Having scaffolded the page, open the `PageModel` file, and comment out both the existing `OnGetAsync` and `OnPostAsync` methods. Once you have done that, replace them with the following implementations:

```
public IActionResult OnGet() => NotFound();
public IActionResult OnPost() => NotFound();
```

I have chosen to return a 404 Not Found message if a user attempts to reach this endpoint either by a `GET` or `POST` request. You can provide an alternative response if you prefer. You might want to redirect the user somewhere else; for example, you can use a `RedirectToPage` method. The important thing is to replace both the `OnGet` and `OnPost` handlers. When disabling other pages that only have an `OnGet` handler, it is still wise to implement the `OnPost` handler as well.

You have completed the task of identifying who your users are, so they are no longer anonymous. Armed with this information, you can now determine what they are authorized to do while on the site. At the moment, you have a management area open to everyone. You don't want just anyone to have access to the property management pages, for example. They could set their vacation price, and that would never do. In the next chapter, you will learn how to use authorization to restrict access to select visitors based on what you know about them.

Summary

- Authentication is the process of identifying site users, so they are no longer anonymous.
- You need to add authentication services and middleware to enable authentication in your application.
- Authentication in a browser-based web application relies primarily on cookies that hold the current user's identity in an encrypted format.